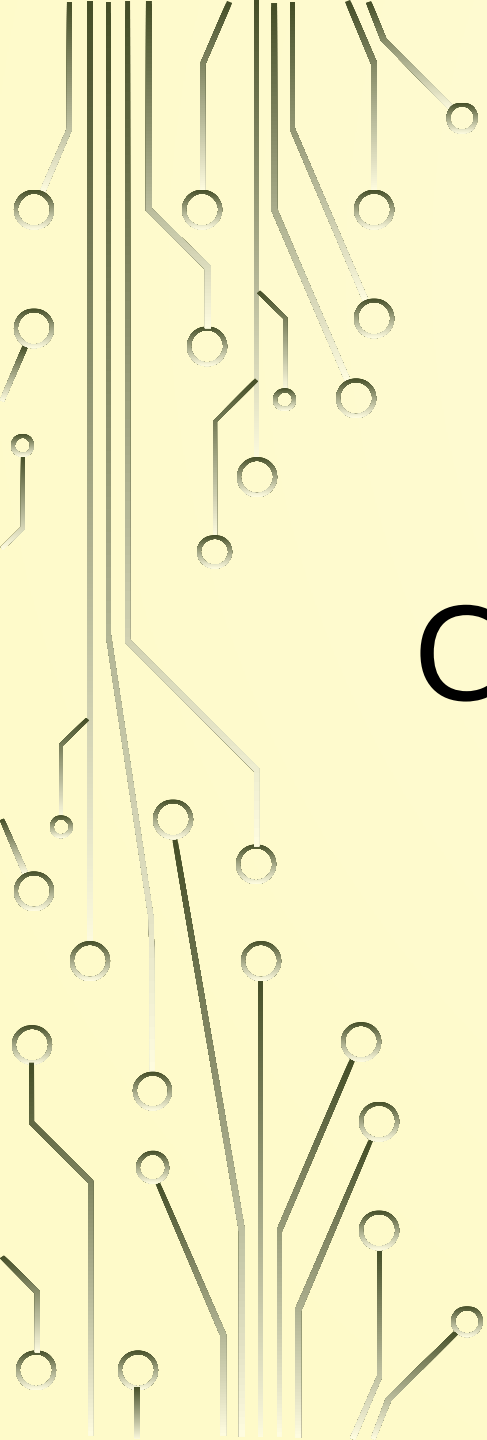


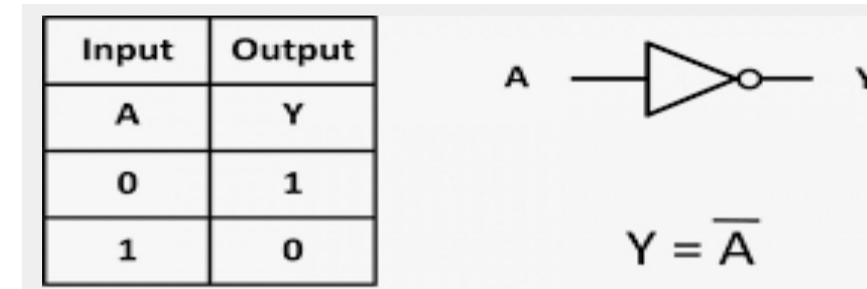
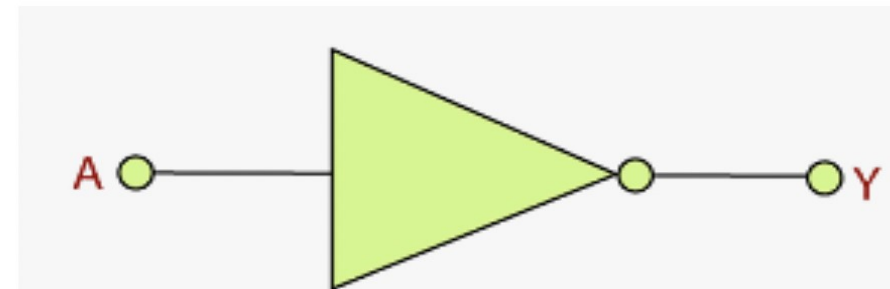
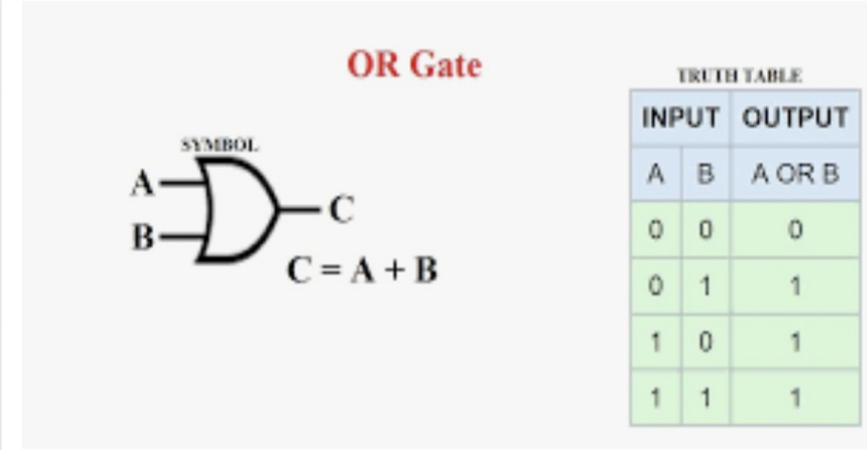
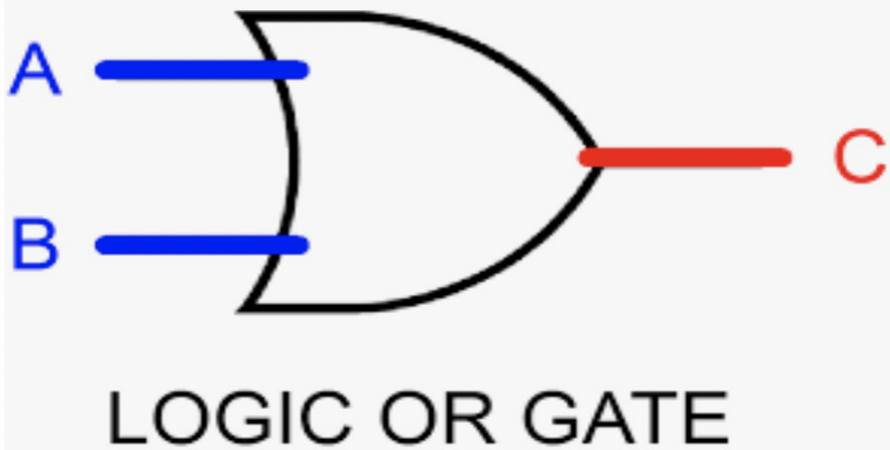
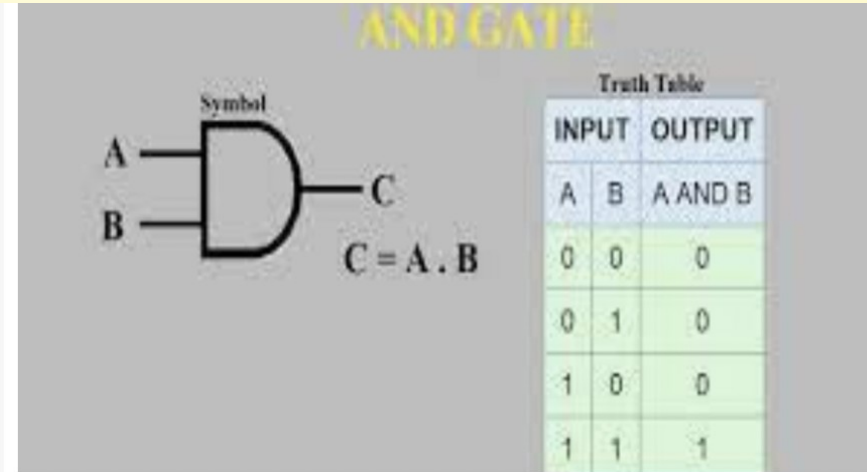
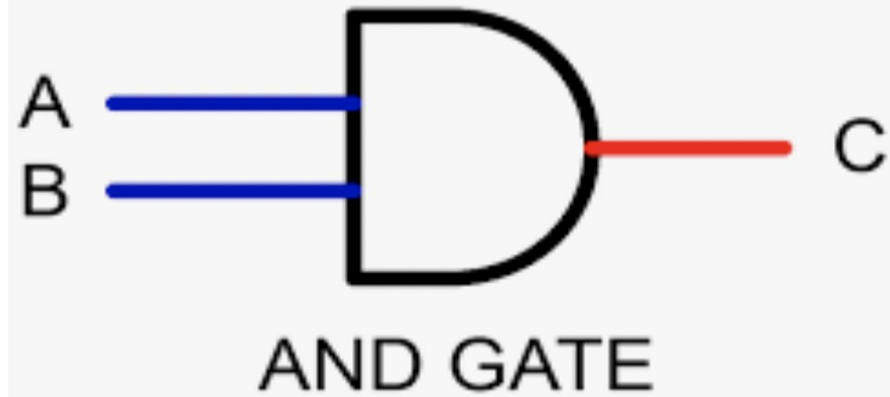
A decorative graphic on the left side of the page, resembling a circuit board or a stylized tree. It consists of numerous thin, dark lines that branch out from the left edge, ending in small circles. The lines are arranged in a way that suggests a complex network or a series of connections.

CHAPTER 3

3.2 BOOLEAN ALGEBRA (1 OF 17)

- Boolean algebra is a mathematical system for the manipulation of variables that can have one of two values.
 - In formal logic, these values are “true” and “false.”
 - In digital systems, these values are “on” and “off,” 1 and 0, or “high” and “low.”
- Boolean expressions are created by performing operations on Boolean variables.
 - Common Boolean operators include AND, OR, and NOT.

LOGICAL GATES



3.2 BOOLEAN ALGEBRA (2 OF 17)

- A Boolean operator can be completely described using a truth table.
- The truth table for the Boolean operators AND and OR are shown at the right.
- The AND operator is also known as a **Boolean product**. The OR operator is the **Boolean sum**.

X AND Y

X	Y	XY
0	0	0
0	1	0
1	0	0
1	1	1

X OR Y

X	Y	X+Y
0	0	0
0	1	1
1	0	1
1	1	1

3.2 BOOLEAN ALGEBRA (3 OF 17)

- The truth table for the Boolean NOT operator is shown at the right.
- The NOT operation is most often designated by a prime mark ($\mathbf{x'}$). It is sometimes indicated by an overbar ($\overline{\mathbf{x}}$) or an “elbow” ($\neg\mathbf{x}$).

NOT X	
X	X'
0	1
1	0

3.2 BOOLEAN ALGEBRA (5 OF 17)

- The truth table for the Boolean function:

$$F(x, y, z) = xz' + y$$

is shown at the right.

- To make evaluation of the Boolean function easier, the truth table contains extra (shaded) columns to hold evaluations of subparts of the function.

$$F(x, y, z) = xz' + y$$

x	y	z	z'	xz'	xz' + y
0	0	0	1	0	0
0	0	1	0	0	0
0	1	0	1	0	1
0	1	1	0	0	1
1	0	0	1	1	1
1	0	1	0	0	0
1	1	0	1	1	1
1	1	1	0	0	1

3.2 BOOLEAN ALGEBRA (6 OF 17)

- As with common arithmetic, Boolean operations have rules of precedence.
- The NOT operator has highest priority, followed by AND and then OR.
- This is how we chose the (shaded) function subparts in our table.

$$F(x, y, z) = xz' + y$$

x	y	z	z'	xz'	xz' + y
0	0	0	1	0	0
0	0	1	0	0	0
0	1	0	1	0	1
0	1	1	0	0	1
1	0	0	1	1	1
1	0	1	0	0	0
1	1	0	1	1	1
1	1	1	0	0	1

3.2 BOOLEAN ALGEBRA (7 OF 17)

- Digital computers contain circuits that implement Boolean functions.
- The simpler that we can make a Boolean function, the smaller the circuit that will result.
 - Simpler circuits are cheaper to build, consume less power, and run faster than complex circuits.
- With this in mind, we always want to reduce our Boolean functions to their simplest form.
- There are a number of Boolean identities that help us to do this.

3.2 BOOLEAN ALGEBRA (16 OF 17)

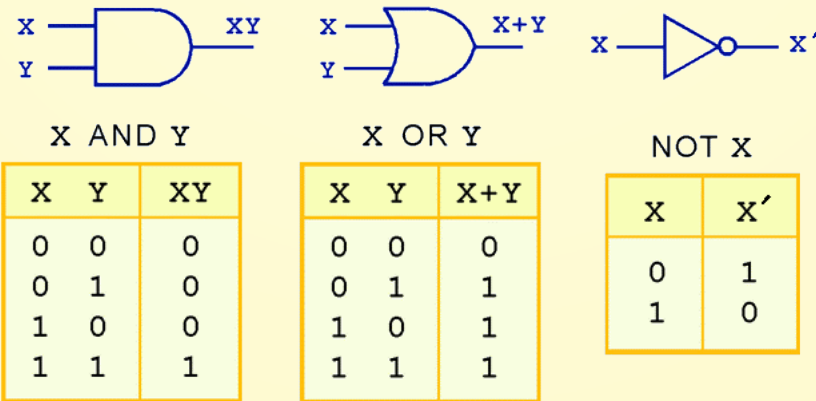
- It is easy to convert a function to sum-of-products form using its truth table.
- We are interested in the values of the variables that make the function true ($= 1$).
- Using the truth table, we list the values of the variables that result in a true function value.
- Each group of variables is then ORed together.

$$F(x, y, z) = xz' + y$$

x	y	z	$xz' + y$
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

3.3 LOGIC GATES (2 OF 6)

- The three simplest gates are the AND, OR, and NOT gates.

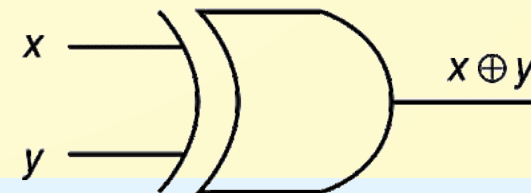


- They correspond directly to their respective Boolean operations, as you can see by their truth tables.

3.3 LOGIC GATES (3 OF 6)

- Another very useful gate is the exclusive OR (XOR) gate.
- The output of the XOR operation is true only when the values of the inputs differ.

X XOR Y		
X	Y	$X \oplus Y$
0	0	0
0	1	1
1	0	1
1	1	0



Note the special symbol $\oplus$ for the XOR operation.

3.3 LOGIC GATES (4 OF 6)

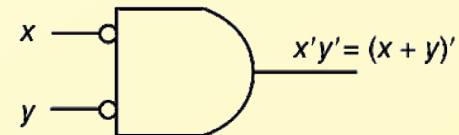
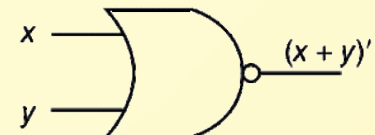
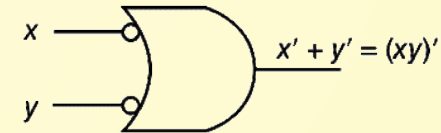
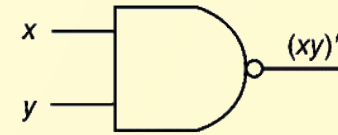
- NAND and NOR are two very important gates. Their symbols and truth tables are shown at the right.

X NAND Y

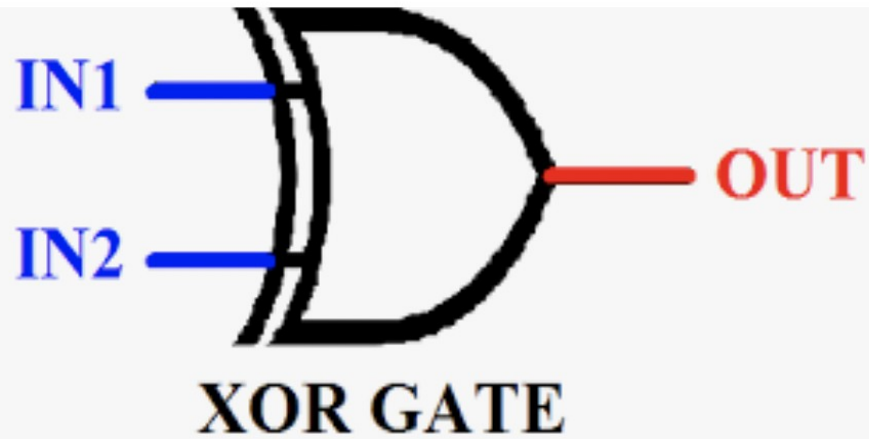
X	Y	X NAND Y
0	0	1
0	1	1
1	0	1
1	1	0

X NOR Y

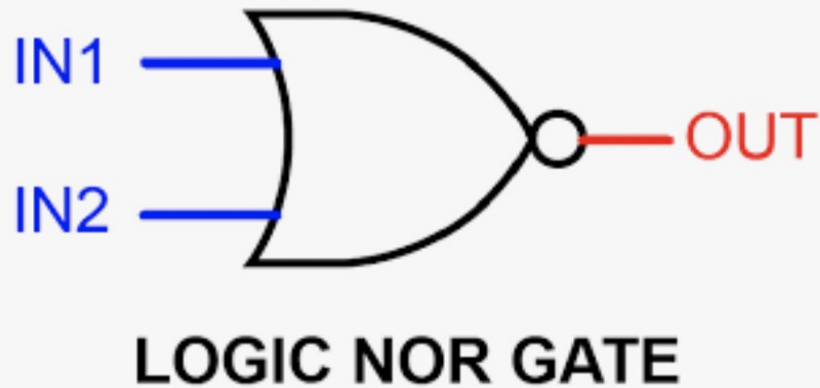
X	Y	X NOR Y
0	0	1
0	1	0
1	0	0
1	1	0



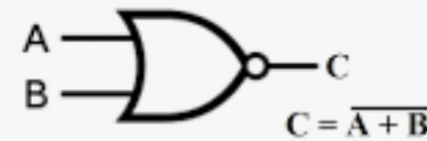
XOR , NAND, NOR GATES



A	B	Out
0	0	0
0	1	1
1	0	1
1	1	0



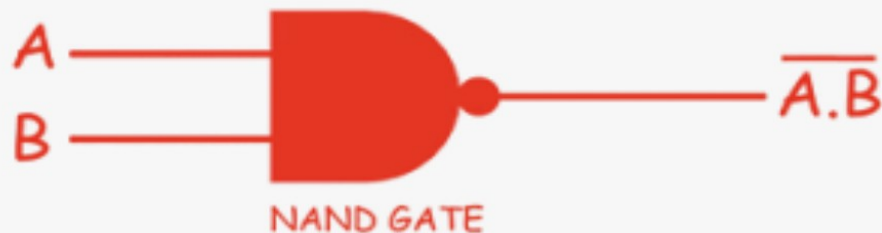
NOR GATE



TRUTH TABLE

INPUT		OUTPUT
A	B	A NOR B
0	0	1
0	1	0
1	0	0
1	1	0

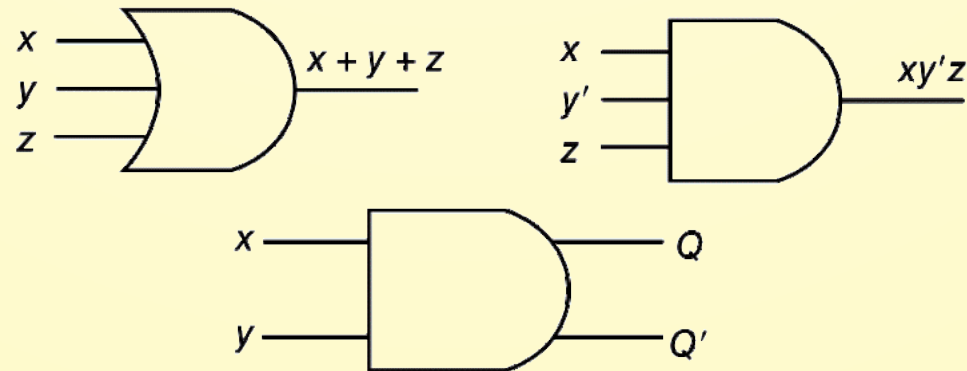
ProjectIoT123.com



Input		Output
A	B	$Y = \overline{A.B}$
0	0	1
0	1	1
1	0	1
1	1	0

3.3 LOGIC GATES (6 OF 6)

- Gates can have multiple inputs and more than one output.
 - A second output can be provided for the complement of the operation.
 - We'll see more of this later.



CUIRCUT DESIGN?

Suppose we are given the task of designing a logic circuit to help us determine the best time to plant our garden. We investigate three possible factors: (1) time, where 0 represents day and 1 represents evening; (2) moon phase, where 0 represents not full and 1 represents full; and (3) temperature, where 0 represents 45 °F and below, and 1 represents over 45 °F. These three items represent our inputs. After significant research, we determine that the best time to plant a garden is during the evening with a full moon (temperature does not appear to matter). This results in the following truth table:

Time (x)	Moon (y)	Temperature (z)	Plant?
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

3.4.2 DESCRIPTION OF KMAPS AND TERMINOLOGY (1 OF 5)

- A Kmap is a matrix consisting of rows and columns that represent the output values of a Boolean function.
- The output values placed in each cell are derived from the minterms of a Boolean function.
- A *minterm* is a product term that contains all of the function's variables exactly once, either complemented or not complemented.

MINTERMS ?

MINTERM

		YZ			
		00	01	11	10
WX	00	$W'X'YZ$	$W'X'Y'Z$	$W'X'YZ$	$W'X'YZ'$
	01	$W'XY'Z'$	$W'XY'Z$	$W'XYZ$	$W'XYZ'$
	11	$WXY'Z'$	$WXY'Z$	$WXYZ$	$WXYZ'$
	10	$WX'Y'Z'$	$WX'Y'Z$	$WX'YZ$	$WX'YZ'$

MINTERM

		YZ			
		00	01	11	10
X	0	$X'Y'Z'$	$X'Y'Z$	$X'YZ$	$X'YZ'$
	1	$XY'Z'$	$XY'Z$	$X'YZ$	XYZ'

3.4.2 DESCRIPTION OF KMAPS AND TERMINOLOGY (2 OF 5)

- For example, the minterms for a function having the inputs x and y are $x'y$, $x'y$, xy' , and xy .
- Consider the Boolean function, $F(x, y) = xy + xy'$
- Its minterms are:

Minterm	X	Y
$x'y'$	0	0
$x'y$	0	1
xy'	1	0
xy	1	1

3.4.2 DESCRIPTION OF KMAPS AND TERMINOLOGY (3 OF 5)

- Similarly, a function having three inputs, has the minterms that are shown in this diagram.

Minterm	X	Y	Z
$X'Y'Z'$	0	0	0
$X'Y'Z$	0	0	1
$X'Y Z'$	0	1	0
$X'Y Z$	0	1	1
$X Y'Z'$	1	0	0
$X Y'Z$	1	0	1
$X Y Z'$	1	1	0
$X Y Z$	1	1	1

3.4.2 DESCRIPTION OF KMAPS AND TERMINOLOGY (5 OF 5)

- As another example, we give the truth table and KMap for the function, $F(x,y) = x + y$ at the right.
- This function is equivalent to the OR of all of the minterms that

~~$F(x,y) = x + y$~~ have a value of 1. Thus:

$$x'y + xy' + xy$$

$$F(X, Y) = X + Y$$

X	Y	X+Y
0	0	0
0	1	1
1	0	1
1	1	1

X \ Y	0	1
	0	1
0	0	1
1	1	1

3.4.3 KMAP SIMPLIFICATION FOR TWO VARIABLES (1 OF 3)

- Of course, the minterm function that we derived from our Kmap was not in simplest terms.
 - That's what we started with in this example.
- We can, however, reduce our complicated expression to its simplest terms by finding adjacent 1s in the Kmap that can be collected into groups that are powers of two.
- In our example, we have two such groups.
 - Can you find them?

x \ y	0	1
0	0	1
1	1	1

3.4.3 KMAP SIMPLIFICATION FOR TWO VARIABLES (2 OF 3)

- The best way of selecting two groups of 1s form our simple Kmap is shown below.
- We see that both groups are powers of two and that the groups overlap.
- The next slide gives guidance for selecting Kmap groups.

x \ y	0	1
0	0	1
1	1	1

3.4.3 KMAP SIMPLIFICATION FOR TWO VARIABLES (3 OF 3)

- The rules of Kmap simplification are:
 - Groupings can contain only 1s; no 0s.
 - Groups can be formed only at right angles; diagonal groups are not allowed.
 - The number of 1s in a group must be a power of 2 – even if it contains a single 1.
 - The groups must be made as large as possible.
 - Groups can overlap and wrap around the sides of the Kmap.

3.4.4 KMAP SIMPLIFICATION FOR THREE VARIABLES (1 OF 7)

- A Kmap for three variables is constructed as shown in the diagram below.
- We have placed each minterm in the cell that will hold its value.
 - Notice that the values for the yz combination at the top of the matrix form a pattern that is not a normal binary sequence.

		yz			
		00	01	11	10
x	0	$x'y'z'$	$x'y'z$	$x y z$	$x'y z'$
	1	$x y'z'$	$x y'z$	$x y z$	$x y z'$

3.4.4 KMAP SIMPLIFICATION FOR THREE VARIABLES (2 OF 7)

- Thus, the first row of the Kmap contains all minterms where x has a value of zero.
- The first column contains all minterms where y and z both have a value of zero.

x	yz			
	00	01	11	10
0	$x'y'z'$	$x'y'z$	xyz	$x'yz'$
1	$xy'z'$	$xy'z$	xyz	xyz'

MINTERM (3 & 4) VARIABLES

MINTERM

YZ		00	01	11	10
WX	00	$W'X'YZ$	$W'X'Y'Z$	$W'X'YZ$	$W'X'YZ'$
	01	$W'XY'Z'$	$W'XY'Z$	$W'XYZ$	$W'XYZ'$
	11	$WXY'Z'$	$WXY'Z$	$WXYZ$	$WXYZ'$
	10	$WX'Y'Z'$	$WX'Y'Z$	$WX'YZ$	$WX'YZ'$

MINTERM

YZ		00	01	11	10
X	0	$X'YZ$	$X'Y'Z$	$X'YZ$	$X'YZ'$
	1	$XY'Z'$	$XY'Z$	XYZ	XYZ'

3.4.4 KMAP SIMPLIFICATION FOR THREE VARIABLES (3 OF 7)

- Consider the function:
- Its Kmap is given below.
 - What is the largest group of 1s that is a power of 2?

x \ yz	yz			
	00	01	11	10
0	0	1	1	0
1	0	1	1	0

$$F(X, Y, Z) = X'Y'Z + X'YZ + XY'Z + XYZ$$

3.4.4 KMAP SIMPLIFICATION FOR THREE VARIABLES (4 OF 7)

- This grouping tells us that changes in the variables x and y have no influence upon the value of the function: They are irrelevant.
- This means that the function,

x \ yz	yz			
	00	01	11	10
0	0	1	1	0
1	0	1	1	0

reduces to $F(x) = z$.

You could verify this reduction with identities or a truth table.

$$F(x, y, z) = x' y' z + x' y z + x y' z + x y z$$

3.4.4 KMAP SIMPLIFICATION FOR THREE VARIABLES (5 OF 7)

- Now for a more complicated Kmap. Consider the function:

x \ yz	yz			
	00	01	11	10
0	1	1	1	1
1	1	0	0	1

- Its Kmap is shown below. There are (only) two groupings of 1s.

$$F(X,Y,Z) = X'Y'Z' + X'Y'Z + X'YZ \\ + X'YZ' + XY'Z' + XYZ'$$

3.4.4 KMAP SIMPLIFICATION FOR THREE VARIABLES (6 OF 7)

- In this Kmap, we see an example of a group that wraps around the sides of a Kmap.
- This group tells us that the values of x and y are not relevant to the term of the function that is encompassed by the group.
 - What does this tell us about this term of the function?

What about the green group in the top row?

x \ yz	yz			
	00	01	11	10
0	1	1	1	1
1	1	0	0	1

3.4.4 KMAP SIMPLIFICATION FOR THREE VARIABLES (7 OF 7)

- The green group in the top row tells us that only the value of x is significant in that group.
- We see that it is complemented in that row, so the other term of the reduced function is $\mathbf{x'}$
- Our reduced function is $\mathbf{F(x,y,z) = x' + z'}$

Recall that we had six minterms in our original function!

x \ yz	yz			
	00	01	11	10
0	1	1	1	1
1	1	0	0	1

3.4.5 KMAP SIMPLIFICATION FOR FOUR VARIABLES (2 OF 4)

- We have populated the Kmap shown below with the nonzero minterms from the function:

WX \ YZ	YZ			
	00	01	11	10
00	1	1		1
01				1
11				
10	1	1		1

- Can you identify (only) three groups in this Kmap?

Recall that groups can overlap.

$$F(W, X, Y, Z) = W'X'Y'Z' + W'X'Y'Z + W'X'YZ'$$

$$+ W'XYZ' + WX'Y'Z' + WX'Y'Z + WX'YZ'$$

3.4.5 KMAP SIMPLIFICATION FOR FOUR VARIABLES (3 OF 4)

- Our three groups consist of:
 - A purple group entirely within the Kmap at the
 - A pink group that wraps the top and bottom.
 - A green group that spans the corners.
- Thus we have three terms in our final function:

WX \ YZ	00	01	11	10
00	1	1		1
01				1
11				
10	1	1		1

$$F(W, X, Y, Z) = X'Y' + X'Z' + W'YZ'$$

3.4.7 SUMMARY (1 OF 2)

- Kmaps provide an easy graphical method of simplifying Boolean expressions.
- A Kmap is a matrix consisting of the outputs of the minterms of a Boolean function.
- In this section, we have discussed 2-, 3-, and 4-input Kmaps. This method can be extended to any number of inputs through the use of multiple tables.

3.4.7 SUMMARY (2 OF 2)

- Recapping the rules of Kmap simplification:
 - Groupings can contain only 1s; no 0s.
 - Groups can be formed only at right angles; diagonal groups are not allowed.
 - The number of 1s in a group must be a power of 2 – even if it contains a single 1.
 - The groups must be made as large as possible.
 - Groups can overlap and wrap around the sides of the Kmap.
 - Use don't care conditions when you can.

3.5 DIGITAL COMPONENTS (6 OF 8)

- Suppose we are to design a logic circuit to determine the best time to plant a garden.
- We consider three factors (inputs):
 - (1) time, where 0 represents day and 1 represents evening;
 - (2) moon phase, where 0 represents not full and 1 represents full; and
 - (3) temperature, where 0 represents 45°F and below, and 1 represents over 45°F.
- We determine that the best time to plant a garden is during the evening with a full moon.

3.5 DIGITAL COMPONENTS (7 OF 8)

- This results in the following truth table:

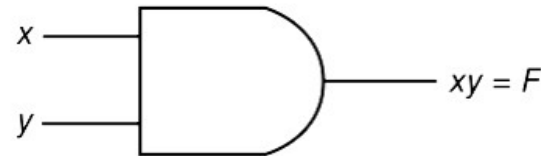
Time (x)	Moon (y)	Temperature (z)	Plant?
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

3.5 DIGITAL COMPONENTS (8 OF 8)

- From the truth table, we derive the circuit:

Time (x)	Moon (y)	Temperature (z)	Plant?
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

$$F(x,y,z) = xyz' + xyz = xy$$



3.6 COMBINATIONAL CIRCUITS (1 OF 12)

- We have designed a circuit that implements the Boolean function:

$$F(X, Y, Z) = X + Y'Z$$

- This circuit is an example of a *combinational logic circuit*.
- Combinational logic circuits produce a specified output (almost) at the instant when input values are applied.
 - In a later section, we will explore circuits where this is not the case.

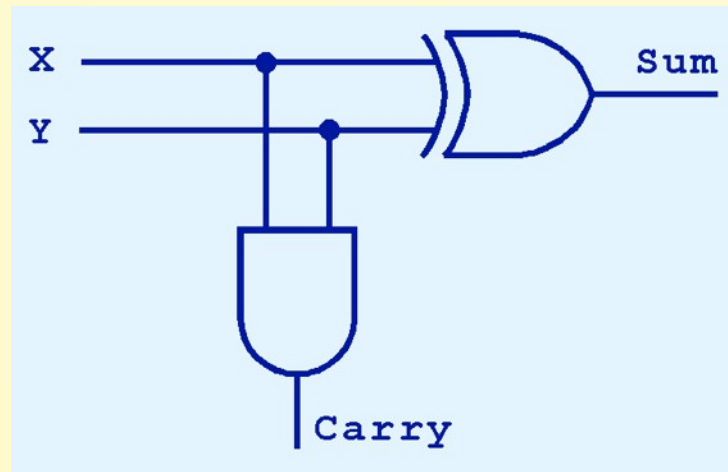
3.6 COMBINATIONAL CIRCUITS (2 OF 12)

- Combinational logic circuits give us many useful devices.
- One of the simplest is the *half adder*, which finds the sum of two bits.
- We can gain some insight as to the construction of a half adder by looking at its truth table, shown at the right.

Inputs		Outputs	
X	Y	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

3.6 COMBINATIONAL CIRCUITS (3 OF 12)

- As we see, the sum can be found using the XOR operation and the carry using the AND operation.



Inputs		Outputs	
X	Y	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

3.6 COMBINATIONAL CIRCUITS (4 OF 12)

- We can change our half adder into to a full adder by including gates for processing the carry bit.
- The truth table for a full adder is shown at the right.

Inputs			Outputs	
X	Y	Carry In	Sum	Carry Out
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1